Agile Software Development

CS40006

UNIT-I

Introduction

Dr. Raghunath Dey

Software development

What is Software Development?

1. Software de-velopment is defined as the process of **designing, creating, testing, and maintaining computer programs and applications**.
2. Software development plays an important role in our daily lives. It not only empowers smart applications but also supports businesses worldwide.
3. Software developers develop the software, which itself is a set of instructions in order to perform a specific task.
4. Developers develop system software, programming software, and application software.

Types of software development : Broad classification

Sequential (plan-driven) methodologies : traditional, linear process. These follow a step-by-step process, where each phase completes before the next begins. Useful when requirements are clear and unlikely to change. Often chosen when scope, timeline, and requirements are well-understood upfront, or when frequent changes are not expected

- **Waterfall Model** : all planning, design, implementation, testing, deployment happen in a fixed linear order. Changes are hard once phases are done.
- **V- Model** : similar to Waterfall but emphasizes testing at every stage, pairing development phases with associated testing phases.
- **Spiral Model** : combines iterative development with risk analysis: project proceeds through repeated cycles (spirals), refining requirements and design iteratively especially for complex / high-risk projects.
- **Incremental Model** : the system is built in increments (modules/features) one after another, so at least parts of the system are usable early.
- **Rapid Application Development (RAD)** : emphasises quick delivery via prototyping, parallel development of modules, early user involvement. Good when requirements are fairly clear and modularization is possible.

Agile and other iterative/fast-cycle methodologies : more modern, flexible process

Under Agile-style/iterative methodologies, there are multiple frameworks and models. According to Itransition, some of the widely used ones are Scrum, kanban, scrumban, scaled agile method, FDD, DSDM which we will study further in detail.

Example

Let’s say a company wants to build a food-delivery app.

- Traditional Development (Waterfall-like)
- They first create a huge document of all requirements: login, restaurant list, payment, offers, GPS tracking, reviews, wallet, notifications etc
- Only after months of development do they release the full product.

Problems:

- By the time it launches, market trends may have changed.
- The customer might realize the initial idea was incomplete.
- If users dislike the UI, changing it becomes expensive.
- Competitors might already launch better features.

Agile will solve our problems !

move quickly, easily

designing, creating, testing, and maintaining computer programs and applications to meet user needs

Agile Software Deve

Imagine going on a trip with friends and a solid plan for day 1-4, but a lot of problems arise upon starting, like it starts raining or the train got late.

If you strictly follow the original plan, the trip becomes stressful. So what do you do?

You start adapting.
You make small plans daily.
You decide together as you go.
You try something, see if it works, and then adjust.
You keep the trip enjoyable by responding to real situations, not forcing old plans.

This is exactly what Agile does in software development.

Traditional software methods behave like the *original rigid travel plan* where everything is decided upfront, and once something changes, the entire plan collapses.

Agile behaves like the *adjustable, daily decision-making during the trip* as you break the work into small parts, take continuous feedback, deliver something usable quickly, and adapt as reality changes.

Agile teaches teams to:

- Work in small, meaningful steps
- Accept that change is normal
- Collaborate closely
- Deliver value continuously
- Measure progress realistically

So just like your smartly-managed trip becomes more enjoyable and successful, Agile helps software teams build better products by staying flexible, practical, and customer-focused.

Agile Software Development

What is Agile Software Development?

- **Agile Software Development** is a Software Development Methodology that values flexibility, collaboration, and customer satisfaction.
- It is based on the **Agile Manifesto**, a set of principles for software development that prioritize individuals and interactions, working software, customer collaboration, and responding to change.
- Agile Software Development is an **iterative and incremental approach to Software Development that emphasizes the importance of delivering a working product quickly and frequently.**
- It involves close collaboration between the development team and the customer to ensure that the product meets their needs and expectations.

In Agile, the same app is built in **small usable pieces**:

Sprint 1: Basic login + browsing restaurants
Sprint 2: Add ordering + cart
Sprint 3: Add payment
Sprint 4: Add GPS + tracking
Sprint 5: Notifications, offers, improvements

After every sprint:

- The team shows a working product to the client
- The client gives feedback
- Requirements change naturally
- The next sprint adapts accordingly

This way:

- The app is released faster
- Changes are welcomed, not feared
- Users get useful features early
- Quality improves continuously
- The product evolves with real customer needs

So in simple terms, Agile Software Development is not just a methodology.
It is a mindset and a way of building software that welcomes change, involves customers closely, and delivers value step-by-step rather than all at once

Manifesto for Agile Software Development

At the end of the 1990s, the software world was in chaos. Projects were collapsing everywhere and developers were stressed, companies were losing money, and customers were constantly disappointed. Big organizations were trying to build software the same way they built bridges: with long, rigid plans that couldn't survive the real world.. In February 2001, frustrated by this mess, seventeen of the world's smartest software thinkers decided to escape to a snowy ski resort in Snowbird, Utah. They didn't go there for skiing; they went because they were tired, annoyed, and desperate to fix how software was being built. These seventeen people all believed in different lightweight methods like Scrum, XP, Crystal , but they all agreed that the traditional system was broken. Over cups of coffee, heated arguments, laughter, and long walks in the cold, something unexpected happened: they realized they shared the same core beliefs. They sat down together and wrote four simple lines that captured everything they felt about building good software. These four lines became the **Agile Manifesto**. It wasn't born in a boardroom or a lab but it was born out of frustration, honesty, and a desire to make software development human again. And from that frozen mountain, a revolution began, one that changed how the entire world builds technology today.

4 Agile Values (Agile Manifesto)

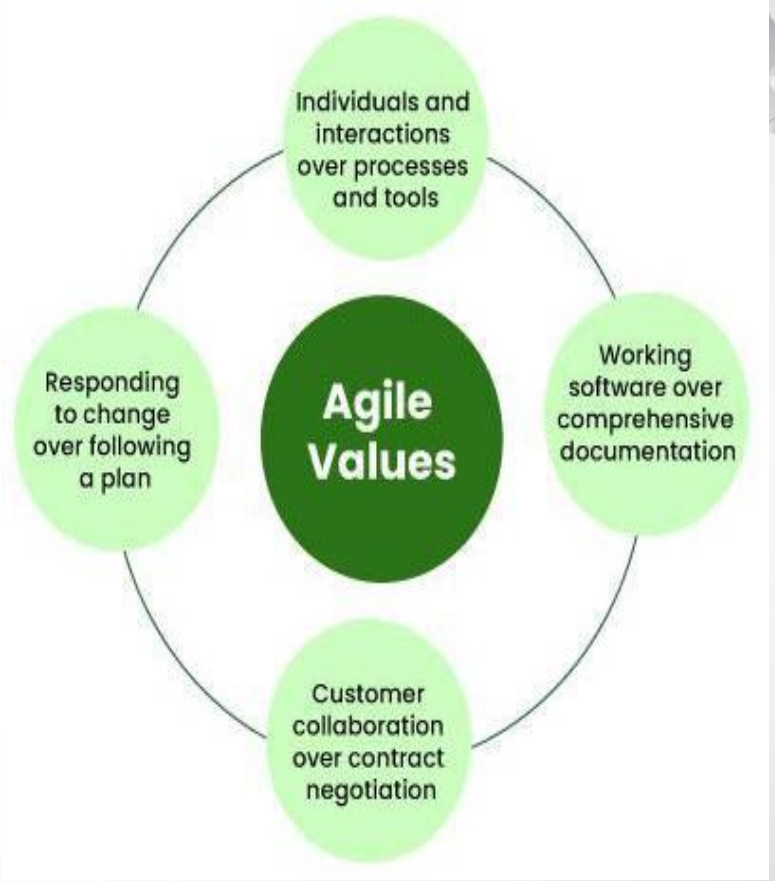
- The four core values of Agile software development, as outlined in the Agile Manifesto, focus on what truly matters for creating successful software.

1. **Individuals and Interactions over Processes and Tools:** This value stresses that the strength of the team and how well they work together is more important than the tools or processes they use. Of course, tools and processes help, but the real success of a project comes from good teamwork, open communication, and collaboration.

2. **Working Software over Comprehensive Documentation:** Agile prefers delivering functional software quickly rather than getting down in lengthy documentation. While some documentation is needed, the focus is on getting a working product into the hands of the user and improving it based on feedback.

3. **Customer Collaboration over Contract Negotiation:** Agile values regular collaboration with customers over sticking strictly to contracts. The idea is to involve the customer throughout the development, verifying that the product meets their needs and making adjustments based on their feedback.

4. **Responding to Change over Following a Plan:** In Agile, change is expected, and the approach encourages flexibility. Rather than rigidly following a plan that may no longer apply, Agile teams adapt and adjust based on new information, changing market conditions, or including customer requirements.



12 Principles of Agile Software Development

- There are 12 agile principles mentioned in the Agile Manifesto.
- **Agile principles are guidelines for flexible and efficient software development.**
- They emphasize frequent delivery, embracing change, collaboration, and continuous improvement.
- The focus is on delivering value, maintaining a sustainable work pace, and ensuring technical excellence.

Agile Manifesto Principles

1	Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.	4	Business people and developers must work together daily throughout the project.
2	Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.	5	Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.
3	Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.	6	The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.

Agile Manifesto Principles

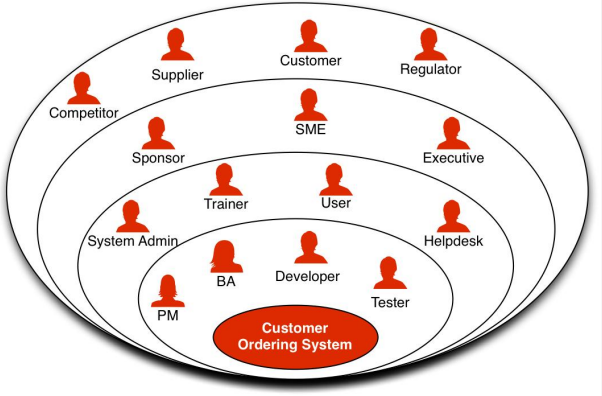
7	Working software is the primary measure of progress.	10	Simplicity – the art of maximizing the amount of work not done – is essential.
8	Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.	11	The best architectures, requirements, and designs emerge from self-organizing teams.
9	Continuous attention to technical excellence and good design enhances agility.	12	At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

Stakeholders

- In any software development project especially Agile , there are many *stakeholders*. A stakeholder is **any person who is affected by the product or who can affect the product**. They may be Users , Developers, Managers, Sponsors, Customers, Legal teams, Operations teams, Even the public or media
- Different stakeholders have different interests, expectations, and priorities.
- **The onion model of stakeholders** explains how different people involved in a software project can be arranged in layers, just like the layers of an onion.

  At the center of the onion is the **product**, which is the most important part of the project. The first layer around it contains the **primary stakeholders**, such as the users, support staff, and system administrators. These people directly use or operate the product and therefore influence it the most. The second layer contains the **secondary stakeholders**, like technical managers, the user management team, sales, purchasing, and the legal team. They are not directly using the product but are involved in decisions that affect how the product is built and used in the business. The outermost layer consists of the **tertiary stakeholders**, which include sponsors, clients, regulators, suppliers, the public, and even the media.

  These people are not part of the daily development work, but they impact things like funding, approval, compliance, and public acceptance of the final product. Thus, the onion model helps us understand how close or far each stakeholder is from the product and how much influence they have on the project.



Example

Imagine the team is building a new feature: **“Order Tracking from Restaurant to Home”**

Primary Stakeholders:

- ➔ **Users**: Want live tracking
- ➔ **Delivery Partners**: Need accurate map
- ➔ **System Admins**: Manage backend servers

Secondary Stakeholders:

- **Sales Team**: Use data to promote premium delivery
- **Legal Team**: Ensures privacy rules (GPS data usage)
- **Technical Managers**: Approve architecture, APIs

Tertiary Stakeholders:

- **Sponsors**: Fund the new tracking system
- **Regulators**: Check data protection compliance
- **Public/Media**: Give feedback on features

Most involved stakeholders:

- **Product Owner**: Decides priority of live-tracking
- **Developers**: Build map, GPS, routes
- **Project Manager**: Coordinates budget + deadlines
- **End Users**: Test and provide feedback

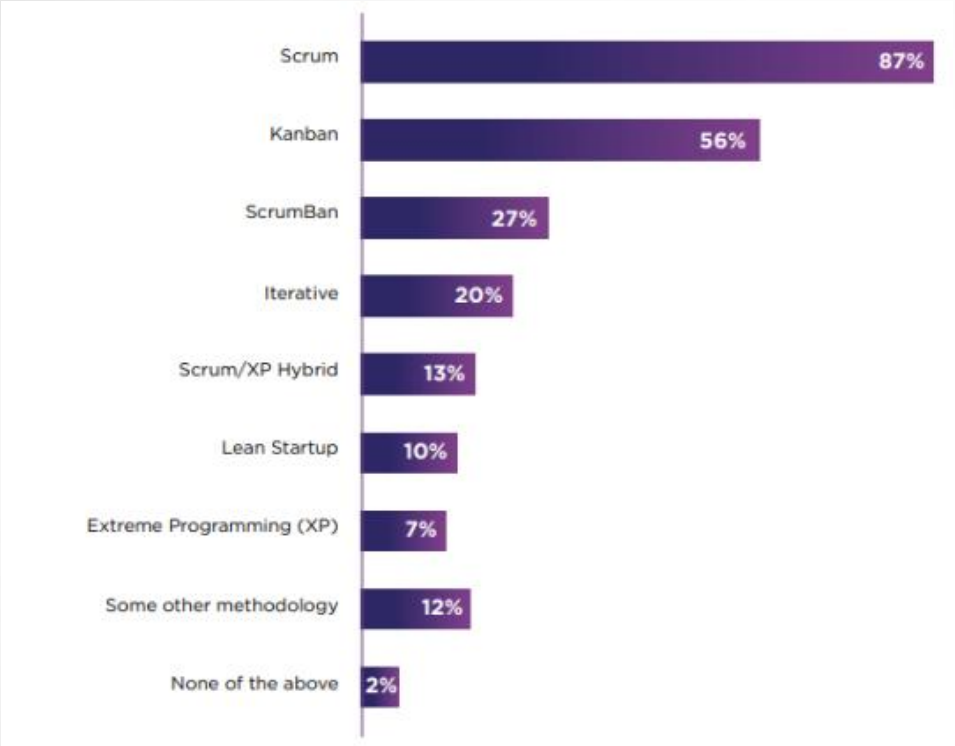
Challenges in Agile

Although Agile is flexible and effective, teams often face several challenges while implementing it. One major difficulty is dealing with **changing requirements**, which can cause rework if not properly managed. Agile also needs **continuous customer involvement**, and lack of timely feedback can delay progress. Another challenge is maintaining **effective communication** within the team, especially in distributed or remote environments. Agile teams must also be **self-organizing**, which is difficult when members lack experience or clarity in roles. Finally, Agile requires continuous testing and frequent delivery, which may stress the team if proper planning and resources are not available.

- Frequent requirement changes may cause confusion or rework.
- Requires strong customer involvement; delays impact progress.
- Communication gaps can break the Agile flow.
- Self-organizing teams may struggle without skilled members.
- Continuous testing and delivery increase workload and pressure.

Top Agile Methodologies in Software Development

- The following table overview some commonly used and well-documented frameworks for agile software development. Many organizations adopt and modify them to fit the needs of their agile processes.
- **Scrum -** uses short sprints (1–4 weeks) with defined roles like Product Owner, Scrum Master, and Development Team. It includes events such as Daily Scrum and Sprint Review to ensure regular progress and feedback, delivering small increments of working software.
- **Kanban -** uses a visual board and WIP limits to manage a continuous flow of work. Tasks move across columns as they progress, helping teams reduce bottlenecks and improve speed without fixed iterations or required roles.
- **Scrumban -** combines Scrum's sprint planning with Kanban's flow and WIP limits. It gives teams structure when needed while allowing continuous movement of tasks, making it flexible and easy to adapt
- **Lean Software Development -** focuses on eliminating waste, improving efficiency, and delivering only what adds value. It promotes continuous improvement and avoids unnecessary work, delays, or extra features.
- **DSDM (Dynamic Systems Development Method) -** is time-boxed and iterative, guided by clear principles. It aims to deliver essential features quickly, prioritizing requirements based on business value and ensuring active user involvement
- **XP (Extreme Programming) -** emphasizes strong engineering practices like pair programming, TDD, and continuous integration. It releases small updates frequently to maintain high quality and adapt quickly to changes.
- **Crystal -** adjusts its processes based on project size and criticality. It focuses on communication, collaboration, and frequent delivery while remaining lightweight and highly adaptable.
- **FDD (Feature-Driven Development) -** builds the system through short iterations based on small, client-valued features. Its structured five-step approach ensures clear progress, quality, and timely delivery.



Based on State of Agile 2022 report

Agile teams typically use frameworks as a starting point for their transformation and customize elements to suit their specific requirements. Each framework has its own set of pros and cons, and what works for one team may not work for another.

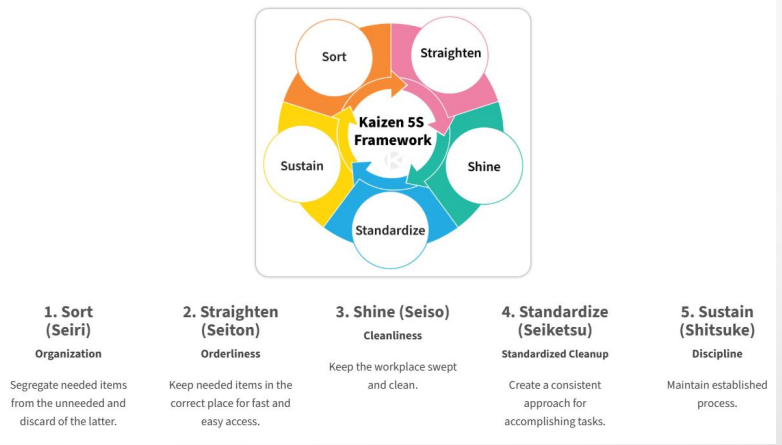
Lean Approach (Kaizen, Kanban, Waste Management)

Lean is all about **delivering value efficiently by minimizing waste**. Agile often incorporates Lean principles to improve workflow and quality. Think of it like cleaning your study table, when books, cups, chargers, papers are scattered, you **waste time** finding what you need but after you clean it, work becomes smoother.

Lean = cleaning the process so work flows smoothly

Kaizen -

- ➔ A Japanese business philosophy, of continuous improvement, where company employees at all levels, work together to achieve regular, incremental improvements to the manufacturing process. It focuses on applying small, daily changes that result in major improvements over time. Kaizen has most commonly been associated with manufacturing operations, as used in Toyota, but it has also been successfully adopted in non-manufacturing environments. With the continuous strive for efficiency and growth, many companies in various industries have followed Toyota's lead and started practicing the Kaizen philosophy.
- ➔ It is continuous Improvement where small, incremental improvements in processes, products, and team efficiency.
- ➔ Encourages learning from every iteration.



Kanban

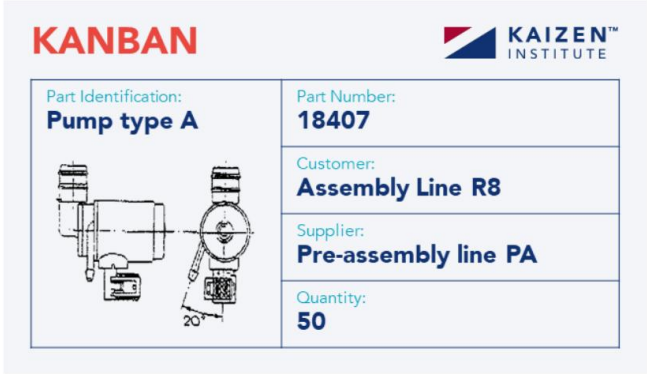
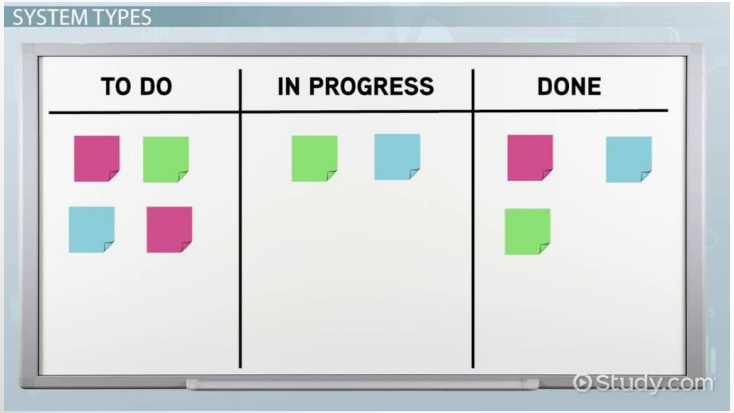


Figure 1 – Example of a production Kanban



- Kanban means “card” or “visual signal” in Japanese, reflecting the essence of the system: clear and visual communication of production or transportation needs, promoting a pull system where production occurs only based on actual customer demand.
- Kanban can take different forms, being implemented both physically and digitally.
- Additionally, it can be applied in various contexts: production Kanban, which is used to signal production orders, and logistics or movement Kanban, which is used to signal material transportation orders.
- Visual Workflow Management or Visual board showing :

To Do → In Progress → Done.

- Helps track work, manage bottlenecks, and improve flow.
- For example, you place “Revise Agile” in the *To Do* column, “Coding Practice” in *In Progress*, and “Make Notes” in *Done*, and then move each note forward as you work. This makes everything clearly visible, so there’s no confusion or forgetting tasks. Agile teams use Kanban for the same reason so that everyone can instantly see who is doing what, where the work is stuck, and it prevents people from taking too many tasks at once by using WIP limits.

Waste Management

They avoid building useless features like “dark mode,” “share attendance on social media,” until the customer actually asks. Thus, they save time and deliver a clean product quickly. Lean identifies and removes waste as anything that doesn’t add value to the customer.

Core principle: Identify and eliminate "Muda," which is the Japanese term for any activity that consumes resources but does not add value for the customer.

Common Types of Waste:

- Extra features not needed
- Waiting time
- Frequent hand-offs
- Defects & rework
- Multitasking
- Over-planning
- Unnecessary documentation

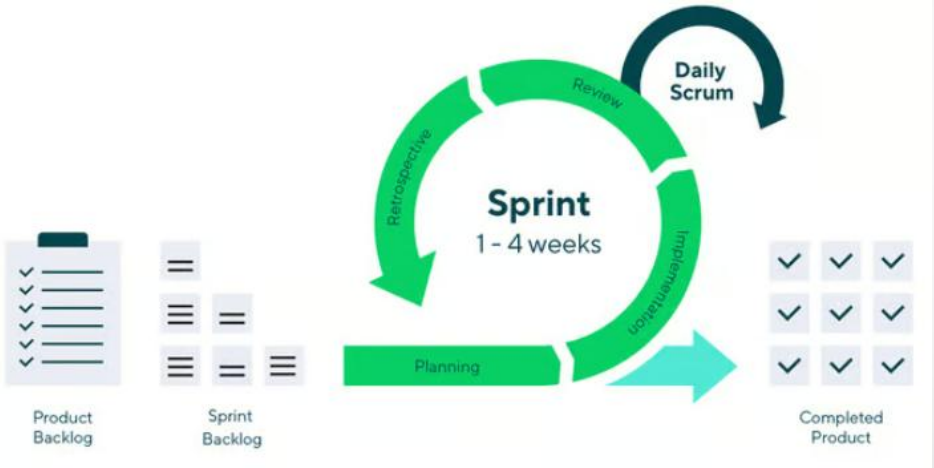
Goal: Deliver only what matters, in the cleanest and fastest way. Lean techniques focus on identifying and removing the seven main types of waste (over production, waiting, transport, inventory, over-processing, motion, and defects) from the process. Eliminating these wastes from the process reduces overall operation costs and optimizes the process.

Traditional vs Agile Software Development

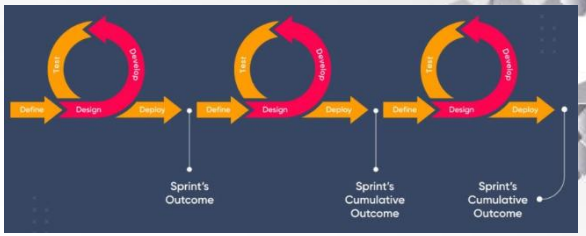
Feature / Aspect	Traditional Development	Agile Development
Approach	Linear, sequential	Iterative, incremental
Requirements	Fixed at the beginning	Can change anytime
Customer Involvement	Only at start & end	Continuous & frequent
Planning	Heavy upfront planning	Adaptive planning each sprint
Delivery	Final product delivered at end	Working product delivered every iteration
Flexibility	Low — changes are costly	High — changes are welcomed
Team Structure	Hierarchical	Self-organizing & cross-functional
Documentation	Heavy documentation	Minimal but sufficient documentation
Testing	After development	Continuous during sprint
Risk Handling	High risk (issues found late)	Low risk (early detection)
Feedback	Late feedback	Continuous feedback
Quality	Improved at the end	Improved throughout
Transparency	Low	High
When to Use	Stable requirements	Dynamic requirements

Agile Software Development Cycle

- Agile is a flexible, iterative approach to software development that prioritizes collaboration, rapid prototyping, and continuous improvement.
- **Iteration -** A short, fixed time period (usually 1–4 weeks) where the team builds a small working piece of the software.
- **Scrum** is a popular Agile framework where work is divided into **sprints**, and the team follows specific roles
- A **sprint** is a timeboxed iteration (usually 2 weeks) in Scrum where a set of features is planned, developed, tested, and delivered.
- Extreme Programming (XP) is an Agile method that focuses heavily on engineering excellence, such as: Test-Driven Development (TDD) , Pair Programming, Continuous Integration, Refactoring . The Iterations in XP are usually shorter than Scrum (sometimes 1 week).
- **This will be studied in more details in further units.**



Agile Software Development Cycle



Agile Software Development Life Cycle

1. Pre-Planning

- Define product vision, scope, goals
- Form team, gather initial requirements
- Create the Product Backlog
- Rough estimate of time, cost, features

2. Planning

- Decide sprint length, iteration schedule
- Estimate development speed
- Discuss major modules & user stories

3. Release Planning

- Break backlog into releases
- Each release contains multiple sprints
- Rough timeline for each release

4. Iteration / Sprint Planning

- Select user stories for the sprint
- Break stories into tasks
- Estimate each task
- Decide sprint goa

5. Sprint Execution

- Daily Scrum meetings
- Design → Code → Test → Integrate
- Continuous review & collaboration

6. Sprint Review

- Demonstrate working software to customer
- Collect feedback
- Identify changes

7. Sprint Retrospective

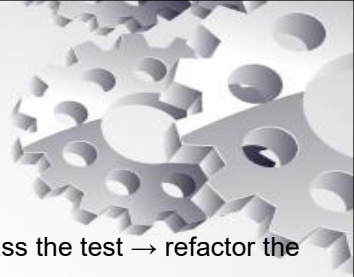
- Discuss what worked / what didn't
- Improve teamwork, tools, and process

8. Backlog Refinement

- Update priorities
- Add new requirements
- Re-estimate timelines

- ❑ Agile development breaks the project into small cycles called **iterations or sprints.**
- ❑ Each sprint delivers a working piece of software that customers can see, test, and give feedback on.
- ❑ Requirements can evolve at any time, and the team adapts every sprint to meet changing business needs.
- ❑ Agile teams are cross-functional and self-organizing, with continuous communication.
- ❑ Testing happens throughout the sprint, reducing errors early.
- ❑ Agile provides better transparency, faster delivery, higher customer satisfaction, and better control over risks, making it ideal for modern, dynamic projects.

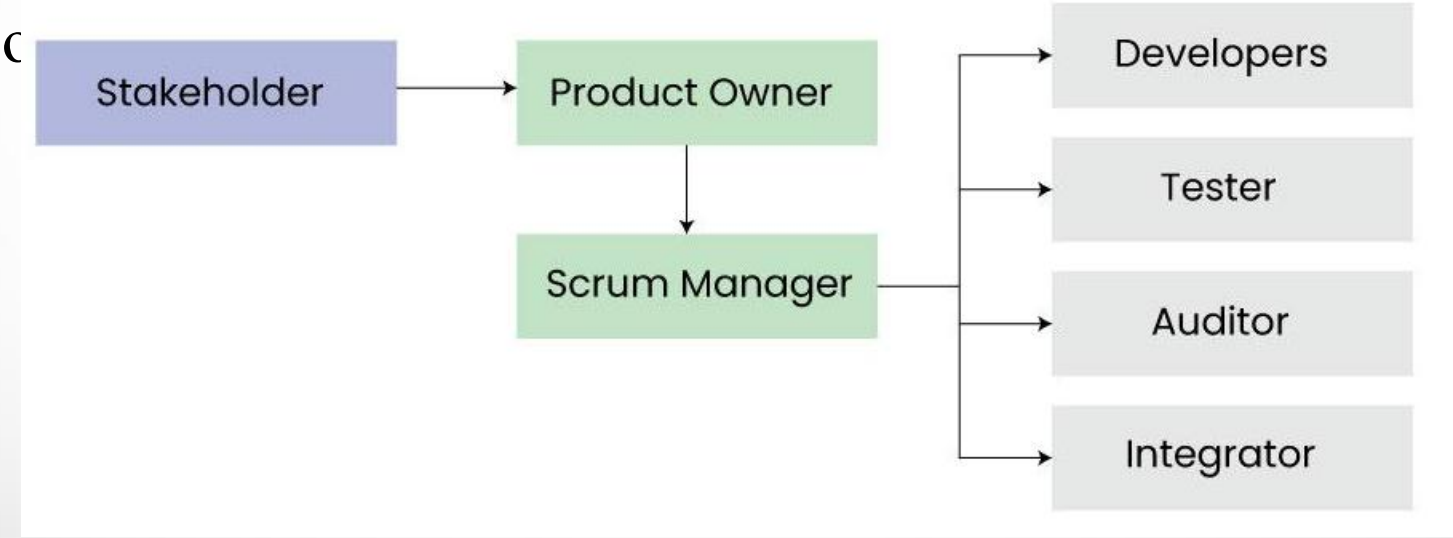
Quality Development in Agile



- **Test-Driven Development (TDD) -** follows a **test-first** mindset: write a failing test → write code to pass the test → refactor the code. This ensures features are built only to meet actual requirements, reduces defects, and keeps the design simple.
- **Behavior-Driven Development (BDD) -** extends TDD by writing executable specifications in natural language. Acceptance criteria follow: **Given** (initial condition) → **When** (action) → **Then** (expected result). This ensures clarity between developers, testers, and customers and improves acceptance-quality.
- **Pair Programming -** Two developers work together on one machine—one writes code (driver), the other reviews each step (navigator). This leads to cleaner design, fewer bugs, shared knowledge, and constant code review.
- **Definition of Done (DoD) -** is a shared checklist that confirms when a task is truly complete. It may include: code done, reviewed, tested, documented, integrated.This ensures consistent quality across all features.
- **Peer Review-** Team members review each other's code and provide feedback. This strengthens code quality, prevents defects, and allows shared responsibility in the team.
- **Trunk-Based Development -** Teams work on small branches that merge back into the **main trunk** quickly or even ideally daily. This prevents merge conflicts, reduces integration failures, and encourages small, safe updates.Often used with CI/CD practices.
- **Build and Test Automation -** Code is built frequently, ideally on every commit, and automated tests run as part of the build. Static/dynamic analysis tools help detect issues early. Automation ensures faster feedback and higher reliability.
- **Continuous Integration (CI) -** Developers integrate their changes frequently (multiple times a day). Every integration triggers automated builds and tests, preventing last-minute integration problems. CI ensures the system is always in a working state.
- **Collective Code Ownership -** The entire team owns all code—anyone can improve any part. This increases collaboration, innovation, defect detection, and avoids dependency on individual members.

Roles Related to the Agile Lifecycle

Agile assigns clear roles throughout the lifecycle so that planning, development, testing, and delivery remain smooth and value-driven. These [illegible] flow from c



1. **Product Owner (PO)** - The product owner is responsible for representing the stakeholders and the end customers. They define the product vision and roadmap, prioritize the product backlog, and accept or reject the resulting work or outcome. They also manage stakeholders expectation, communicate goals, and ensures the team is delivering maximum business value.
2. **Scrum Master:** The scrum master is responsible for managing the agile processes and removing obstacles that prevent the development team from being productive. They facilitate meetings, promote collaboration, and encourage the team to improve its practices. In simple terms, they act as a binding force towards reaching conclusion of the project.
3. **Development Members/Developers:** The developers are a cross-functional group that carries out the actual work to build the product incrementally. They collaborate daily in short, high-paced meetings to analyse, design, develop, test, and implement the user stories from the product backlog. The development team are usually structured yet flexible and self organized while carrying their task execution.
4. **Stakeholders:** Stakeholders are individuals or groups who are impacted by the project. They may be within or external to the organization. They provide feedback on the requirements while reviewing progress of the project and also evaluate the final product. The stakeholders are usually engaged and kept in loop to allow the project team to meet their required business needs.
5. **Integrator:** The integrator is responsible for technical integration of work by the development team. They ensure that the software/hardware components fit together properly. The integrator also makes sure the system meets quality standards and integrates smoothly with external systems.
6. **Independent Auditor and Tester:** The independent tester objectively evaluates the system to ensure the quality of the product. They audit the product for bugs and flaws from an unbiased perspective. The tester identifies defects and works with the team to get them fixed.